

Projektaufgaben 1. Teil

Shared-Memory

Lösungsideen dürfen nicht ausgetauscht werden. Eventuelle inhaltliche oder organisatorische Fragen stellen Sie bitte im Diskussionsforum in Moodle. Achten Sie dabei darauf, nichts über Ihre Lösung zu verraten. Sollte dies in Ausnahmefällen nicht möglich sein, schicken Sie eine E-Mail an r.nather@uni-kassel.de

Bitte geben Sie die Lösungen bis spätestens **26.11.2025, 10:00 Uhr** per E-Mail ab.

a) `broken.c`

Laden Sie das vorhandene OpenMP-Programm `broken.c` von Moodle herunter. Das Programm ist syntaktisch korrekt und lässt sich kompilieren und ausführen. Allerdings haben sich mehrere Stil- und Logik-Fehler bei der Benutzung von OpenMP eingeschlichen. Finden und lösen Sie alle Fehler, sodass die Ergebnisse der parallelen Berechnungen garantiert korrekt sind, die Effizienz ist dabei nicht zu berücksichtigen. Führen Sie im Anschluss das Programm zur Kontrolle mehrfach aus, da manche Fehler nicht bei jeder Ausführung auftreten. Verwenden Sie beim Kompilieren die Optionen `-O0` und `-lm`. Kommentieren Sie jede von Ihnen durchgeführte Änderung im Quellcode kurz.

b) `sierpinski.c`

Laden Sie das Programm `sierpinski.c` von Moodle herunter. Das Programm berechnet ein Bild des sogenannten Sierpinski-Teppichs¹ indem es wiederholt eine Menge von Sample-Punkten transformiert und im Bild einfärbt, bis die Verteilung der Sample-Punkte im Bild sich nicht mehr stark ändert. Dafür benutzt das Programm die folgende Datenstrukturen und Algorithmus:

`pixels` – Bild, `int`-Array der Größe `size × size`

`samples` – Punkte die in den Iterationen transformiert und im Bild gefärbt werden

`diff` – Unterschied zwischen Bild in jetziger und vorhergehender Iteration

Wiederhole bis `diff ≤ max_diff` oder `max_iter` Iterationen erreicht sind:

- Für jeden Punkt (x,y) in `samples`:
 - Wähle zufällig die Abbildungsparameter T aus `trafo`
 - Aktualisiere Sample-Punkt: $(x,y) \leftarrow \text{transform}((x,y), T)$
 - Berechne Bildkoordinaten des Pixels der vom (neuen) Sample-Punkt getroffen wird:
 $(u,v) = (\text{floor}(x \cdot \text{size}), \text{floor}(y \cdot \text{size}))$
 - Inkrementiere `pixels[u][v]`
- Aktualisiere `diff`

Das Programm ist syntaktisch korrekt und lässt sich kompilieren und ausführen, führt jedoch bislang nur eine sequentielle Berechnung durch. Parallelisieren Sie das Programm mit OpenMP. Ändern Sie dabei nicht das Abbruchkriterium.

Hinweis: Obwohl die Verwendung von `(s)rand` üblicherweise nicht thread-safe ist, kann das in dieser Aufgabe ignoriert werden.

¹<https://de.wikipedia.org/wiki/Sierpinski-Teppich>

c) `heatmap_analysis.c(pp)`

Schreiben Sie ein effizientes OpenMP-Programm, das ein zweidimensionales Array analysiert, welches als sogenannte *Heatmap* interpretiert wird. Berücksichtigen Sie dabei alle in der Vorlesung behandelten Effizienzaspekte.

Legen Sie ein `unsigned long`-Array der Größe `rows * columns` an. Zur Vergleichbarkeit der Ergebnisse muss das Array wie folgt initialisiert werden (Ihr C/C++-Code darf hiervon abweichen, sofern das Ergebnis identisch ist):

```
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < columns; ++j) {
        srand(seed * concatenate(i, j));
        A[i][j] = rand() % (upper - lower) + lower;
    }
}
```

Die Implementierung der Methode `concatenate` ist auf der letzten Seite des Aufgabenblatts vorgegeben. Die Initialisierung darf sequentiell erfolgen. Wenn ja, müssen Sie sicherstellen, dass jedem `A[i][j]` der gleiche Wert wie oben zugewiesen wird. `rows`, `columns`, `seed`, `lower` und `upper` sind Programmparameter (siehe unten).

Die Analyse besteht aus zwei Teilaufgaben:

Teil 1, Bereichssummen/sliding sums: Für jede Spalte x soll die maximale Summe eines vertikalen Fensters der Höhe `window_height`, im weiteren als h bezeichnet, bestimmt werden. Es ist jeweils die Summe über das Intervall $[i, i + h - 1]$ für $i = 0$ bis `rows` $- h$ zu berechnen.

Teil 2, Lokale Hotspots: Ein Wert `A[i][j]` gilt als lokaler Hotspot, wenn er strikt größer ist als alle seine direkten Nachbarn (oben, unten, links, rechts). Berechnet werden soll die die Anzahl lokaler Hotspots für jede Zeile.

Achtung: Jeder Wert, der in der Analyse verwendet wird, muss `work_factor`-mal transformiert werden, bevor er in Berechnungen einfließt. Die Transformation erfolgt über die Methode `hash`, welche auf der letzten Seite des Aufgabenblatts vorgegeben ist. Beispiel:

```
for (int w = 0; w < work_factor; ++w)
    A[i][j] = hash(A[i][j]);
```

Das Programm soll am Ende folgende Informationen ausgeben:

- Die Maximalwerte der Bereichssummen pro Spalte (wenn `verbose=1`),
- die Anzahl der Hotspots pro Zeile (wenn `verbose=1`) bzw. die Gesamtsumme (immer),
- die gesamte Ausführungszeit (immer).

Die Ausgabe muss exakt wie in den zugehörigen Beispielen formatiert sein. Ob die Ausgabe selbst parallelisiert wird, bleibt Ihnen überlassen. Die Berechnungen müssen immer durchgeführt werden; unabhängig davon, auf was `verbose` gesetzt ist.

Programmaufruf:

```
./heatmap_analysis columns rows seed lower upper window_height  
verbose num_threads work_factor
```

columns	Anzahl der Spalten im Array
rows	Anzahl der Zeilen im Array
seed	Initialwert für den Zufallszahlengenerator
lower, upper	Wertebereich für Zufallszahlen (untere inklusive, obere exklusiv)
window_height	Fensterhöhe für die vertikale Bereichsanalyse
verbose	bei 1 wird das Array, die Maximalwerte der Bereichssummen pro Spalte, und die Anzahl der Hotspots pro Zeile ausgegeben. Bei 0 wird nur die Gesamtsumme der Hotspots ausgegeben
num_threads	Anzahl der OpenMP-Threads
work_factor	Anzahl der Transformationen pro Wert

Beispiel 1:

```
./heatmap_analysis 3 4 42 0 10 2 1 1 1  
Starting heatmap_analysis  
Parameters: columns=3, rows=4, seed=42, lower=0, upper=10,  
            window_height=2, verbose=1, num_threads=1, work_factor=1  
  
A:  
3, 6, 7  
6, 3, 7  
2, 5, 0  
4, 8, 9  
Max sliding sums per column:  
8075180050492084533, 17804482663663377439, 12434629885917944900  
  
Hotspots per row:  
Row 0: 0 hotspot(s)  
Row 1: 0 hotspot(s)  
Row 2: 1 hotspot(s)  
Row 3: 1 hotspot(s)  
Total hotspots found: 2  
Execution took 0.0020 s
```

Beispiel 2:

```
./heatmap_analysis 1024 786 1337 0 100 20 0 1 10  
Starting heatmap_analysis  
Parameters: columns=1024, rows=786, seed=1337, lower=0, upper=100,  
            window_height=20, verbose=0, num_threads=1, work_factor=10  
Total hotspots found: 157565  
Execution took 0.6218 s
```

Beispiel 3:

```
./heatmap_analysis 1024 786 123 0 100 55 0 1 100
Starting heatmap_analysis
Parameters: columns=1024, rows=786, seed=123, lower=0, upper=100,
           window_height=55, verbose=0, num_threads=1, work_factor=100
Total hotspots found: 156972
Execution took 1.0078 s
```

Speedup-Messung: Ermitteln Sie für Ihr Programm Speedups für 2, 4, 8, 16, 24, 32, 48 Threads. Führen Sie die Messungen auf dem ITS-Cluster durch und geben Sie die Messwerte sowie Speedup-Werte mit ab (inklusive eines grafischen Plots). `verbose` soll auf 0 gestellt werden, die restlichen Parameter sollen geeignet gewählt werden. Die Zeitmessung soll unmittelbar nach dem Einlesen der Kommandozeilenparameter beginnen und unmittelbar nach der Ausgabe enden.

Abschlussbemerkungen:

Eventuelle Fragen zu den Aufgaben stellen Sie bitte im Moodle-Diskussionsforum.

Cluster: Alle Programme müssen auf dem ITS-Cluster der Uni Kassel ausführbar sein. Für die Speedup-Messungen muss der Compiler `gcc/14.2.0` und die Partition `pub23` verwendet werden.

Abgabe: Die Lösungen müssen bis spätestens 26.11.2025, 10:00 Uhr per E-Mail an fohry@uni-kassel.de mit dem Betreff "EinfPV-OpenMP-Abgabe" gesendet werden. Später abgegebene Lösungen werden als nicht abgegeben gewertet. Bitte geben Sie zu jedem Programm die vollständigen Kommandozeilen zur Kompilierung und Ausführung (inklusive eventuell zu setzender Umgebungsvariablen), und die Slurm-Skripte zum Starten auf dem Cluster mit ab. Bitte benennen Sie die Dateien wie angegeben.

concatenate:

```
unsigned concatenate(unsigned x, unsigned y) {
    unsigned pow = 10;
    while (y >= pow)
        pow *= 10;
    return x * pow + y;
}
```

hash:

```
unsigned long hash(unsigned long x) {
    x ^= (x >> 21);
    x *= 2654435761UL;
    x ^= (x >> 13);
    x *= 2654435761UL;
    x ^= (x >> 17);
    return x;
}
```